

Penetration Testing Report

Secure Document Sharing System

CS 419: Secure Web Application Project

Team: Alina Nesimi & Mina Odabasi

1. Executive Summary

This penetration test analyzes the Secure Document Sharing System (SDSS), a web application built using the Flask framework, that allows users to securely upload, store, and share documents. The system supports user authentication, role based access control, storage of encrypted documents, document sharing, version control, an audit trail, as well as security and audit logs.

This penetration test evaluated the application across several areas which included authentication, authorization, input validation, session security, and security configuration. The app demonstrated strong security measures and safeguards, such as protected storage using Fernet, password complexity enforcement, account lockout, input control, and document and audit control. It also included built in session protections and application security controls.

There were no significant vulnerabilities found and the controls seemed to be functioning as intended. Although there were some minor issues detected, such as session handling limitations and guest access design considerations, however this does not pose a critical risk to the system.

2. Testing Checklist

A. Authentication Testing

- **Brute force protection**
 - Tracks failed login attempts using a failed_attempts counter
 - Temporary locks account after 5 consecutive failed login attempts
 - Implements IP-based rate limiting to restrict repeated login attempts

- **Password complexity enforcement**
 - Has password minimum (12 characters) and a max length
 - Password Requires:
 - Uppercase letters
 - Lowercase letters
 - Numbers
 - Special characters
- **Session management**
 - Generates a secure session token after successful login
 - Stores session token in a cookie with security flags
 - Validates session token on every request to maintain authentication
 - Implements session timeout based on inactivity → time out for 30 min
- **Logout functionality**
 - Destroys the session on logout
 - Removes session token from server
 - Clears session cookie from the user's browser
- **Password reset security**
 - Only admin users can reset passwords
 - Resets failed attempts and lock out time for the account

B. Authorization Testing

- **Horizontal privilege escalation**
 - Users can only access own documents
 - Users can only access other documents if
 - Document was shared with them
 - They are admin
- **Vertical privilege escalation**
 - The `require_role("admin")` decorator help to enforce vertical/elevated permissions
 - If user role is not admin they are not able to perform these actions
- **Direct object reference**
 - Server checks if you are allowed access to a document by checking permission
 - If a user modifies a `document_id` in the URL access is still denied because it checks one's permission and/or if the doc was shared with said user (also must be logged in)
- **Forced browsing**

- You can't just type a in a URL and get in the system first has to check if you're logged in and allowed

C. Input Validation Testing

- **XSS attempts**
 - Use jinja default auto escaping (“{{ }}”) so that malicious code runs as text
- **Path traversal**
 - This is prevented by cleaning filenames, generating your own file names to store, and restricting file storage to go into upload folder.
- **Command injection**
 - Code never runs and system (os) or shell commands therefore users cannot execute tasks as commands
- **File upload vulnerabilities**
 - Check both file content and extensions
 - Sanitize file names
 - Enforces file extension whitelisting
 - Uses MIME to validates file content

D. Session Security Testing

- **Session fixation**
 - After logging in random session token is generated
 - secrets.token_urlsafe(32) makes token random and hard to guess (32 bytes long)
- **Session hijacking**
 - Session is stored on the server rather than browser because
 - Cookies include these flags:
 - httponly → prevents JavaScript access)
 - sameaite=Strict → prevents cross-site attacks
 - secure=not app.debu → ensures cookies only send over https
- **Concurrent session handling**
 - Not limit to how many sessions a user can have
 - A user can be logged in across multiple devices
- **Session timeout**
 - The session will be deleted after inactivity of 30 minutes

E. Configuration Testing

- **Security headers**
 - Security headers used are (as recommended in project instructions):
 - Content Security Policy (CSP)
 - X-Frame-Options (clickjacking protection)
 - X-Content-Type-Options (MIME sniffing prevention)
 - Referrer-Policy
 - Permissions-Policy
 - Strict-Transport-Security (HSTS)
 - X-XSS-Protection
- **TLS/SSL configuration**
 - `ssl_context=("cert.pem", "key.pem")` enables http bys generating key for encryption and self signed certificate
- **Error handling**
 - Return 403 Forbidden if unauthorized access
 - Return 404 Not Found if something is not found
- **Debug mode disabled**
 - For local / development testing debug can be on but disable when app is live

3. Findings

****NO CRITICAL OR HIGH FINDINGS****

Medium Finding: Concurrent session handling is not enforced

- **Description:**
 - Application does not invalidate older sessions for the same user when they log in again. It adds a new token without removing existing tokens for that username.
- **Impact:**
 - This means use can remain active across multiple browsers/devices at same time which can weaken access control.
- **Steps to reproduce:**
 1. Log in with one account in one browser.
 2. Log in with the same account with second device
 3. Notice how your able to be logged in across 2 devices

- **Proof of concept:**

```
# logs user in
def create_session(username):
    sessions = load_json(app.config["SESSIONS_FILE"], {}) # loads existing s
    token = secrets.token_urlsafe(32) # gen random unique token to be used a

    #defines a new session entry in the sessions dictionary using the genera
    # The session data includes the username of the logged-in user, the time
    sessions[token] = {
        "username": username,
        "created_at": time.time(),
        "last_activity": time.time()
    }

    save_json(app.config["SESSIONS_FILE"], sessions)
    log_security_event("SESSION_CREATED", username=username, details={"token
    return token # sends token to browser as a cookie to maintain the user's
```

→ No code handling/ deleting previous sessions

- **Remediation recommendation:**

- Expire older sessions for the same username on new login, or clearly document that multiple simultaneous sessions are allowed.

Low 2: Guest view depends on document ID secrecy

- **Description:**

- The public view route allows access without login if the user is set to "guest"
- This means access depends on if one possesses or is given the document ID.
- But it only allows guests to view the document page.

- **Impact:**

- May not be fully valid way of guest being able to view the actual document however allowing guests to download the document may be security risk

- **Steps to reproduce:**

- Share a document with someone given the "guest" username.
- Visit /view/<document_id> without logging in.

- **Proof of concept:**

```
if "guest" in shared_with:

    log_security_event("GUEST_VIEW", document_id=document_id, details={"ip":
request.remote_addr})

    return render_template("view.html", document=document, events=[],
current_user=None)
```

Directs the user to view the page of the document where they can only see filename owner and version.

- **Remediation recommendation:**
 - Clearly define guest access behavior

4. Conclusion

After completing the penetration testing it demonstrated strong performance from the Secure Document Sharing System. It exemplified secure control mechanisms in place that matched the course project scope. There are strong authentication processes, effective authorization (RBAC) for operations on documents and administrator tasks, prestorage encryption, session control, security logging and audit trail mechanisms. Security headers are available, and TLS/SSL can be incorporated into the system.

Critical and high severity flaws were not found. Only 2 flaws of medium and low severity level were found and consisted of session handling and guest role issues. Overall, the application demonstrates strong effort in secure web development and aligns well with the goals of the assignment.